

KANConvNet Classifier on Unified IoV Datasets

Implementation report — FedIoV (Heidari et al., FGCS 2026)

1. Project context and scope

This report documents the classifier-side implementation of the FedIoV framework proposed by Heidari, Rastegar and Khonsari in *FedIoV: A secure and adaptive federated framework for real-time intrusion detection in vehicular networks* (Future Generation Computer Systems 181, 2026, 108448). FedIoV combines four components: secure model initialisation and encrypted communication, a two-stage robust aggregation pipeline (TOPSIS + Multi-Krum), a Kolmogorov–Arnold convolutional classifier (KANConvNet, paper §3.4), and genetic algorithm-based hyperparameter optimisation.

The work covered here is the **second** of those four components — the local KANConvNet classifier that each vehicle would train on its own data before sending updates to the aggregator. The federation, the robust aggregation, and the GA-based hyperparameter search are not implemented yet and are scoped for the next milestone.

1.1 What is in the deliverable

- A from-scratch PyTorch implementation of KANConvNet matching paper equations 16–19.
- Per-dataset adapters for four benchmark IoV datasets: IEEE VTC–CAN, Car-Hacking, CICIDS-2017, VeReMi.
- A unified data pipeline that reduces every dataset to a common 46-dimensional input vector, so the same classifier definition trains on each.
- Cleanup scripts that strip the features the paper marks as identifier-leaking or unstable (absolute timestamps, port identifiers, raw node IDs, RSSI).
- A training/evaluation loop with per-dataset hyperparameters drawn from paper Table 4.

2. Datasets and cleanup

Four benchmark IoV datasets are used. Each is preprocessed twice: an initial cleanup pass produced the `cleaned_*` CSVs in `Preprocessed_Dataset`; a second pass (`scripts/clean_remaining_features.py`) removed the identifier-leaking or unstable columns that survived the first pass. The script is idempotent — re-running it is a no-op once each file has been cleaned.

Dataset	Native format	Native classes	Columns dropped this pass
IEEE VTC–CAN	HCRL CAN log	4 (Attack-free, DoS, Fuzzy, Impersonation)	“Timestamp: <x>” prefix on every line
Car-Hacking	HCRL CAN (flat CSV + log)	5 (normal, DoS, Fuzzy, RPM, gear)	AbsoluteTimestamp (col 0) on flat CSVs; “Timestamp: <x>” on log

Dataset	Native format	Native classes	Columns dropped this pass
CICIDS-2017	Flow-level CSV (78 numeric + Label)	15 attack families + BENIGN	Destination Port (col 0)
VeReMi	Per-beacon CSV (kinematics + noise)	18 attack types + BENIGN	Unnamed: 0 (pandas index). Raw Node IDs / RSSI were not present — already removed in your earlier preprocessing pass.

Sizes shrank after cleanup. For VeReMi the savings were modest (7.44 GB → 7.26 GB, ~70 s on disk). For the CAN datasets the Timestamp prefix accounted for roughly 40% of each line, so e.g. CAN-VTC Attack_free dropped from 206.7 MB to 121.4 MB.

3. The unified 46-feature representation

The paper's KANConvNet eq. (16) takes $x \in \mathbb{R}^{46}$ as input. To run the same classifier on four datasets with very different native schemas we enforce a 46-dim contract at the adapter layer. The shape is unified; the per-slot semantics are not — slot 0 is a different quantity in each dataset, so we train one model per dataset rather than a single shared model.

3.1 CAN-style datasets (IEEE VTC-CAN, Car-Hacking)

Both share `data/can_features.py`. We slide a length- $W = 64$ window with stride 16 over the message stream and compute the timeless statistics below. Because timestamps were stripped, classical inter-arrival-time features were replaced with entropy and ID-diversity descriptors.

Slots	Feature	Count
0–7	Payload byte means (b0..b7), normalised /255	8
8–15	Payload byte standard deviations	8
16	DLC mean	1
17	DLC standard deviation	1
18	Unique CAN-ID ratio (uniq / W)	1
19	Dominant-ID frequency	1
20–43	CAN-ID histogram over 24 hash buckets	24
44	Mean Shannon entropy across the 8 byte slots	1
45	Non-zero payload byte ratio	1
	Total	46

3.2 CICIDS-2017

Rows are already flow-level feature vectors (78 numeric columns after Destination Port removal). No windowing is needed; we select 46 stable columns spanning the main feature families:

- Flow duration, total forward/backward packets, total lengths
- Per-direction packet length max/min/mean/std (8 columns)
- Flow bytes/sec, packets/sec, flow IAT mean/std/max/min
- Forward IAT total/mean/std/max/min and Backward IAT total/mean/std/max/min
- Forward/backward header length and packet rate
- Min/max/mean/std/variance of packet length
- FIN/SYN/RST/PSH/ACK/URG flag counts
- Down/Up ratio, average packet size

The full list is in `data/cicids_dataset.py` (SELECTED_COLUMNS_46). Labels default to binary BENIGN vs ATTACK; passing `--multi-class` on the CLI enables the 15-attack-family taxonomy.

3.3 VeReMi

Per-beacon CSV with 18 kinematic columns (type, rcvTime, pos × 2, spd × 2, acl × 2, hed × 2, each with a noise channel) plus two label columns. We slide $W = 64$ windows with stride 16 over the rows (streaming in 1-million-row chunks so the 7 GB file fits in memory).

Slots	Feature	Count
0–17	Mean of each of the 18 kinematic columns	18
18–35	Standard deviation of each of the 18 columns	18
36–37	Speed magnitude mean and std ($\sqrt{\text{spd}_0^2 + \text{spd}_1^2}$)	2
38–39	Acceleration magnitude mean and std	2
40–41	Position spread x and y (max – min)	2
42	Mean heading magnitude	1
43–44	rcvTime delta mean and std	2
45	Fraction of position-broadcast messages (type == 3)	1
	Total	46

4. What KANConvNet does internally

The Kolmogorov–Arnold representation theorem states that any continuous multivariate function on a bounded domain can be written as a finite composition of continuous univariate functions plus addition. KAN networks operationalise this by moving the learnable nonlinearity from the nodes (as in a standard MLP) to the edges. Every (input, output) edge carries its own learnable univariate function ϕ , and each output neuron is the sum of its incoming ϕ values. Two practical benefits the paper claims: fewer parameters than an equally expressive MLP, and per-edge functions are plottable, so the model is more interpretable than a black-box deep net.

4.1 Fourier KAN Linear layer

We realise each per-edge ϕ with a truncated Fourier basis of K modes:

$$\phi_{i,j}(x_i) = \sum_{k=1..K} [a_{i,j,k} * \sin(k * x_i) + b_{i,j,k} * \cos(k * x_i)]$$

$$y_j = \text{bias}_j + \sum_i \phi_{i,j}(x_i)$$

Implementation lives in `models/kan_layers.py` (`FourierKANLinear`). The operation is vectorised with `torch.einsum`; there are no Python loops over input or output features.

4.2 Kolmogorov activation

Between the KAN layer and the readout, the paper applies a custom $\sigma_{\text{Kolmogorov}}$ nonlinearity. We implement it as a smooth gated composition with per-channel learnable parameters (`models/kan_layers.py`, `KolmogorovActivation`):

$$\text{sigma_K}(x) = \alpha * \tanh(\beta * x) + \gamma * \text{SiLU}(\delta * x) + \epsilon$$

At initialisation ($\alpha=1$, $\beta=1$, $\gamma=0$, $\epsilon=0$) the layer collapses to plain tanh, so training starts in a well-behaved regime.

4.3 The full forward pass (paper eqs. 16–19)

<code>x</code> in $\mathbb{R}^{46}$	# 46-dim window feature vector
<code>z0 = Preprocess(x)</code>	# z-score + percentile clip (DataLoader)
<code>z1 = W1 . F(z0) + b1</code>	# FourierKANLinear (eq. 16)
<code>z2 = sigma_Kolmogorov(z1)</code>	# KolmogorovActivation (eq. 17)
<code>z3, z4, z5 = Linear+ReLU stack</code>	# hidden refinement
<code>z6 = W5 . sigma_Kolmogorov(z5) + b5</code>	# KAN-style head (eq. 18)
<code>y_hat = softmax(z6) -> argmax</code>	# SGD-trained classifier (eq. 19)

An optional `KANConv1d` backbone (`models/kan_conv.py`) is available for the case where the adapter emits raw windowed sequences ($B, 46, W$) rather than collapsed summary vectors ($B, 46$). It uses the same Fourier-KAN parameterisation but with weight-sharing across time via `torch.nn.functional.unfold`. The current adapters emit ($B, 46$), so the backbone is off by default.

5. End-to-end data flow

Tracing a single sample from raw bytes on disk to a class prediction:

- [1] Raw CSV / CAN log on disk
e.g. `Can_vtc_pro/cleaned_DoS_attack_dataset.csv`
line: ID: 0220 000 DLC: 8 29 c5 26 55 6a 67 02 5d
|
v
- [2] Per-dataset parser (`data/can_vtc_dataset.py`, etc.)
Stream-reads the file row by row, decodes hex bytes,
yields arrays: `can_ids (W,)`, `dlc (W,)`, `payload (W, 8)`
|
v
- [3] Feature extractor (`data/can_features.py`)
Slides a $W=64$ window with stride 16,
computes the 46 statistics described in section 3.1
|
v
- [4] Builder (`data/build_dataset.py`)
Stratified 70/15/15 split, percentile-clip + z-score on train only
Returns three Dataset objects with shape $(N, 46)$
|
v
- [5] DataLoader -> batched tensor $(B, 46)$
|
v
- [6] KANConvNet (`models/kanconvnet.py`)
`z1 = FourierKANLinear(46 -> 128)`
`z2 = KolmogorovActivation(128)`
`hidden = 3 x (Linear 128 -> 128 + ReLU + Dropout)`
`z6 = KolmogorovActivation(128) -> Linear(128 -> n_classes)`
|
v
- [7] CrossEntropyLoss with class-balanced weights
Optimiser step (per-dataset choice from Table 4)
|
v
- [8] Best checkpoint saved to
`checkpoints/kanconvnet_best_<dataset>.pt`

6. Per-dataset hyperparameters (paper Table 4)

`config.DATASET_HPARAMS` exposes the best-found hyperparameter set per dataset.
`cfg.apply_dataset_defaults()` applies the right row automatically when `--dataset` is passed on the CLI.

Dataset	Optimiser	Epochs	Momentum	Dropout	Layer size
can_vtc	RMSprop	40	0.90	0.20	128
car_hack	AdamW	30	0.85	0.25	64

Dataset	Optimiser	Epochs	Momentum	Dropout	Layer size
cicids	SGD	60	0.80	0.30	256
veremi	Nadam	25	0.95	0.45	128

Common hyperparameters are in dataclass TrainConfig: batch size 96, learning rate 5e-4, L2 weight decay 5e-4, 70/15/15 stratified split, seed 2025. These match the value sets in paper Table 3.

7. Project layout

```
D:\LG_IoV\
|-- Preprocessed_Dataset\           # cleaned source CSVs (input)
|   |-- Can_vtc_pro\cleaned_*.csv
|   |-- Car_hack_pro\Car_hack_pro\cleaned_*.csv
|   |-- CICIDS_pro\CICIDS_pro\cleaned_*.csv
|   `-- VeReMi_pro\cleaned_Veremi_final_dataset.csv
`-- Model\
    |-- README.md
    |-- requirements.txt
    |-- config.py                   # all hyperparameters
    |-- train.py                     # training + eval entry point
    |-- models\
    |   |-- kan_layers.py            # FourierKANLinear, KolmogorovActivation
    |   |-- kan_conv.py              # KANConv1d (optional backbone)
    |   `-- kanconvnet.py            # full classifier (eqs 16-19)
    |-- data\
    |   |-- can_features.py          # timeless 46-feature extractor
    |   |-- can_vtc_dataset.py       # IEEE VTC-CAN adapter
    |   |-- car_hack_dataset.py      # Car-Hacking adapter
    |   |-- cicids_dataset.py        # CICIDS-2017 adapter
    |   |-- veremi_dataset.py        # VeReMi adapter (streamed)
    |   `-- build_dataset.py         # unified dispatcher + split
    |-- scripts\
    |   |-- clean_remaining_features.py # second-pass column scrubber
    |   |-- build_report.js           # DOCX report
    |   `-- build_report_pdf.py      # this PDF
    `-- checkpoints\                 # written by train.py
        |-- kanconvnet_best_<dataset>.pt
        `-- training_log_<dataset>.json
```

8. How to run training

Install once:

```
cd D:\LG_IoV\Model
py -m pip install -r requirements.txt
```

Smoke test (1 epoch, capped row count, validates wiring end-to-end):

```
py train.py --dataset can_vtc --smoke
py train.py --dataset car_hack --smoke
```

```
py train.py --dataset cicids    --smoke
py train.py --dataset veremi    --smoke
```

Full training run with the paper's Table-4 hyperparameters:

```
py train.py --dataset can_vtc
py train.py --dataset car_hack
py train.py --dataset cicids
py train.py --dataset cicids --multi-class    # 15-class taxonomy
py train.py --dataset veremi
py train.py --dataset veremi --multi-class    # 18-attack-type taxonomy
```

Per-run outputs land in `Model/checkpoints/`:

- `kanconvnet_best_<dataset>.pt` — best-validation-loss weights, normalisation statistics, class list.
- `training_log_<dataset>.json` — per-epoch train loss, validation loss, accuracy and macro-F1.

After training, the script reloads the best checkpoint and runs the test split, printing accuracy, macro-F1 and a per-class precision/recall/F1 breakdown.

9. Outstanding work

- Federated wrapper: spawn N local clients, each with a KANConvNet instance trained on a non-IID slice of one dataset (Dirichlet $\alpha=0.3$ per paper §5.2).
- Robust aggregation: TOPSIS-based dimensionality reduction followed by Multi-Krum filtering of client updates (paper §3.x).
- Differential privacy: client-level update clipping ($\|\Delta\theta\|_2 \leq 1.0$) and Gaussian noise $\sigma \in \{1.2, 1.6, 2.0\}$ before transmission (paper §5.3).
- GA-based hyperparameter optimisation across heterogeneous clients (paper §4).
- Reporting: end-to-end latency and energy-per-round measurements against the GROW / GPIDS / BCFL / REMD / CIDF baselines (paper §5.3.1 and §5.3.2).